

ONEDRAFT

CAD-AI — Aria Powered

Validation & Coordination Engine

Version 1.0

Status: Implementation Specification

Database: PostgreSQL

API: REST / JSON

API Version: /api/v1

Identifiers: UUID

Primary Model: Versioned OneDraft Project Model

AI Layer: Aria Powered

Upstream Systems: Project Model Engine / Geometry Engine / Compliance & Regulatory Engine

Downstream Systems: Documentation Engine / Redline Engine / Document Package Engine / Acceptance

1. VALIDATION ENGINE PURPOSE

The OneDraft Validation & Coordination Engine is the computational verification layer between the project model and the generated architectural documentation.

Its purpose is to determine whether a specific version of the OneDraft computational project model satisfies:

geometric constraints

project requirements

architectural coordination requirements

accessibility requirements

spatial requirements

building-code requirements

documentation requirements

interdisciplinary coordination requirements

and other rules registered against the project.

The engine does not modify the project model merely because a conflict is detected.

Instead:

```
MODEL
↓
VALIDATION
↓
FINDINGS
↓
ARIA ANALYSIS
↓
PROPOSED RESOLUTION
↓
COMMAND
↓
NEW REVISION
↓
REVALIDATION
```

Validation is therefore an independent computational function rather than an implicit side effect of drawing generation.

2. VALIDATION PRINCIPLE

Every validation result must identify the exact project state against which it was generated.

The minimum provenance chain is:

```
PROJECT_ID
↓
REVISION_ID
↓
MODEL_VERSION_ID
↓
CODE_MANIFEST_ID
↓
VALIDATION_RUN_ID
↓
VALIDATION_RESULT
```

A validation result without this provenance chain is not considered authoritative project state.

3. VALIDATION DOMAINS

The initial validation engine shall support:

```
CODE
GEOMETRY
SPATIAL
ACCESSIBILITY
LIFE_SAFETY
STRUCTURAL_COORDINATION
```

MECHANICAL_COORDINATION
ELECTRICAL_COORDINATION
PLUMBING_COORDINATION
SITE
DOCUMENT
DRAWING
REQUIREMENT
MODEL_INTEGRITY

Additional domains may be added without changing the fundamental validation architecture.

4. VALIDATION LEVELS

Validation operates at four levels.

Level 1 — Object

A single object is evaluated.

Example:

```
door.width >= required_width
```

Level 2 — Relationship

Two or more objects are evaluated together.

Example:

```
door  
↔  
wall
```

Level 3 — System

A group of coordinated objects is evaluated.

Example:

```
egress path  
↓  
corridor  
↓  
exit  
↓  
exterior discharge
```

Level 4 — Project

The complete project state is evaluated.

Example:

building area
occupancy
height
construction type
egress
fire protection
accessibility
site
documentation

5. VALIDATION RUN

A validation run represents one complete execution of one or more validation domains.

Conceptually:

```
VALIDATION_RUN
  ↓
RULE EXECUTION
  ↓
OBJECT EVALUATION
  ↓
FINDINGS
  ↓
SUMMARY
```

A validation run is immutable after completion.

If the model changes, a new validation run must be created.

6. VALIDATION RUN TABLE

Existing table:

validation.validation_runs

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
code_manifest_id UUID
validation_profile_id UUID
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
status VARCHAR(32) NOT NULL
result_count INTEGER DEFAULT 0
error_count INTEGER DEFAULT 0
warning_count INTEGER DEFAULT 0
passed_count INTEGER DEFAULT 0
```

```
summary JSONB
engine_version VARCHAR(64)
execution_hash VARCHAR(128)
created_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_validation_runs_project
idx_validation_runs_revision
idx_validation_runs_model_version
idx_validation_runs_status
```

7. VALIDATION PROFILES

The engine shall support reusable validation profiles.

validation.validation_profiles

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(255) NOT NULL
description TEXT
profile_type VARCHAR(64) NOT NULL
categories JSONB NOT NULL
rule_filters JSONB
severity_policy JSONB
status VARCHAR(32) NOT NULL
version INTEGER NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Examples:

```
RESIDENTIAL_MVP
COMMERCIAL_BASIC
BUILDING_PERMIT
CONCEPTUAL_REVIEW
FULL_COORDINATION
DOCUMENTATION_QA
```

8. VALIDATION RULE REGISTRY

Rules are registered independently from validation runs.

validation.rules

Fields:

```
id UUID PRIMARY KEY
```

```
rule_code VARCHAR(128) NOT NULL
name VARCHAR(255) NOT NULL
description TEXT
domain VARCHAR(64) NOT NULL
rule_type VARCHAR(64) NOT NULL
source_type VARCHAR(64)
source_reference VARCHAR(512)
severity_default VARCHAR(32) NOT NULL
implementation_key VARCHAR(255) NOT NULL
parameters JSONB
applicability JSONB
version INTEGER NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(rule_code, version)
```

The implementation key maps the rule definition to executable validation logic.

9. RULE IMPLEMENTATION BOUNDARY

Rules shall not be embedded directly into arbitrary API handlers.

The engine exposes a controlled interface:

```
Rule
↓
Applicability
↓
Input Resolver
↓
Evaluator
↓
Result Builder
```

Conceptually:

```
class ValidationRule:
    code: str

    def applies(context):
        ...

    def evaluate(context):
        ...
```

This permits the regulatory engine and validation engine to evolve independently.

10. RULE APPLICABILITY

Before execution, every rule is evaluated for applicability.

Example:

occupancy = ASSEMBLY

may activate assembly-specific rules.

A residential-only rule would therefore return:

NOT_APPLICABLE

rather than:

PASS

This distinction must be preserved.

11. RULE STATUS

Initial rule statuses:

DRAFT
ACTIVE
DEPRECATED
DISABLED

Only:

ACTIVE

rules participate in normal validation.

12. VALIDATION RESULT MODEL

The existing validation result table shall be expanded.

validation.validation_results

Fields:

id UUID PRIMARY KEY
validation_run_id UUID NOT NULL
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
rule_id UUID

affected_objects JSONB
category VARCHAR(64) NOT NULL
status VARCHAR(32) NOT NULL
severity VARCHAR(32) NOT NULL
description TEXT NOT NULL
expected_value JSONB
actual_value JSONB
difference_value JSONB
source JSONB
location JSONB
resolution JSONB
created_at TIMESTAMPTZ NOT NULL

13. RESULT STATUSES

Initial result statuses:

PASS
FAIL
WARNING
INFO
NOT_APPLICABLE
ERROR
SKIPPED

A validation engine failure must never be represented as a successful rule evaluation.

14. SEVERITY

Initial severity hierarchy:

CRITICAL
ERROR
WARNING
NOTICE
INFO

Severity is separate from result status.

Example:

status = FAIL
severity = ERROR

or:

status = WARNING
severity = WARNING

15. FINDING IDENTITY

Every finding receives a stable identity derived from:

```
project_id  
revision_id  
rule_id  
affected_objects  
finding_signature
```

The engine generates:

```
finding_hash
```

This allows the system to determine whether a finding is:

```
NEW  
PERSISTING  
RESOLVED  
REGRESSED
```

16. VALIDATION FINDINGS

A dedicated finding table shall track lifecycle across validation runs.

```
validation.findings
```

Fields:

```
id UUID PRIMARY KEY  
project_id UUID NOT NULL  
rule_id UUID  
finding_hash VARCHAR(128) NOT NULL  
category VARCHAR(64) NOT NULL  
severity VARCHAR(32) NOT NULL  
first_detected_run_id UUID NOT NULL  
last_detected_run_id UUID NOT NULL  
status VARCHAR(32) NOT NULL  
affected_objects JSONB  
description TEXT NOT NULL  
resolution_command_id UUID  
resolved_revision_id UUID  
resolved_at TIMESTAMPTZ  
created_at TIMESTAMPTZ NOT NULL  
updated_at TIMESTAMPTZ NOT NULL
```

17. FINDING LIFECYCLE

The lifecycle is:

NEW
↓
OPEN
↓
ACKNOWLEDGED
↓
RESOLUTION_PROPOSED
↓
RESOLUTION_IMPLEMENTED
↓
REVALIDATED
↓
RESOLVED

A finding may instead become:

WAIVED

or:

NOT_APPLICABLE

subject to appropriate authorization.

18. NO SILENT RESOLUTION

A failed validation result cannot disappear merely because a subsequent validation run no longer detects it.

The engine records:

finding resolved

with:

resolved_revision_id
resolved_at
resolution_command_id

This preserves the historical chain.

19. MODEL INTEGRITY VALIDATION

Before regulatory validation begins, the engine must verify model integrity.

Initial checks include:

orphaned objects
invalid parent references
missing levels

- duplicate identifiers
- invalid geometry references
- missing required relationships
- invalid object states
- broken dependency references

If model integrity fails at a blocking level, downstream validation may be prevented from executing.

20. GEOMETRIC VALIDATION

The engine consumes geometry generated by the Geometry Engine.

It must verify:

- closed boundaries
- non-self-intersection
- valid solids
- valid profiles
- object containment
- object intersection
- minimum separation
- clearances
- alignment
- elevation consistency

Geometry validation does not recreate the geometry engine.

It evaluates the geometry engine's authoritative outputs.

21. SPATIAL VALIDATION

Spatial rules evaluate relationships such as:

- room area
- room dimensions
- circulation width
- clearance
- door swing
- fixture clearance
- occupancy density
- travel distance
- vertical circulation

The engine obtains required thresholds from the applicable regulatory manifest or project requirements.

22. ACCESSIBILITY VALIDATION

Accessibility validation shall evaluate applicable requirements including:

- accessible routes
- door clear widths
- maneuvering clearances
- corridor widths
- accessible parking
- washroom clearances
- vertical accessibility
- threshold conditions
- turning spaces
- accessible fixtures

The actual requirements are sourced from the project's applicable jurisdictional rule set.

23. LIFE-SAFETY VALIDATION

The engine shall support computational evaluation of:

- occupant load
- egress capacity
- exit count
- exit separation
- travel distance
- common path
- dead-end conditions
- exit discharge
- fire separations
- fire-resistance requirements
- protected shafts
- stairs
- ramps

The engine reports findings; it does not represent a professional approval.

24. CODE VALIDATION

Code validation uses:

- CODE_MANIFEST
- +
- CODE_RULES
- +
- PROJECT_MODEL

The evaluator resolves:

- applicability

```
threshold
exception
condition
measurement
result
```

The result must preserve the originating rule citation.

25. CODE TRACEABILITY

Every code finding must be traceable to:

```
code_source_id
code_rule_id
code_manifest_id
rule_version
retrieval_timestamp
```

This prevents a generic AI statement from being treated as a regulatory determination without provenance.

26. REQUIREMENT VALIDATION

Project requirements are validated independently of code requirements.

Example:

```
client requirement:
minimum 3 bedrooms
```

The engine may evaluate:

```
actual bedrooms = 2
```

and return:

```
FAIL
```

even if the project is otherwise code-compliant.

27. DESIGN INTENT VALIDATION

Design intent may be represented through structured project requirements.

Examples:

preserve existing tree
maintain exterior envelope
maintain south-facing glazing
retain existing stair
minimum parking count

Aria may translate natural-language intent into structured requirements, but validation executes against the structured representation.

28. COORDINATION ENGINE

The coordination subsystem evaluates whether different project systems remain mutually consistent.

Initial coordination domains:

ARCHITECTURAL
STRUCTURAL
MECHANICAL
ELECTRICAL
PLUMBING
SITE
FIRE_PROTECTION
DOCUMENTATION

29. COORDINATION CONFLICTS

Examples include:

beam intersects duct
duct intersects wall
door intersects fixture
stair conflicts with structure
shaft does not align between levels
plumbing stack shifts unexpectedly
mechanical equipment lacks clearance
ceiling conflicts with ductwork

Each conflict becomes a validation finding.

30. DEPENDENCY GRAPH

The existing:

`model.relationships`

table becomes the basis of the coordination graph.

Conceptually:

```
OBJECT A
↓
RELATIONSHIP
↓
OBJECT B
↓
DEPENDENCY
↓
VALIDATION RULE
```

This allows a modification to identify downstream objects potentially affected by the change.

31. IMPACT ANALYSIS

Before executing an Aria command, the system may perform impact analysis.

Input:

```
command
```

Output:

```
directly affected objects
indirectly affected objects
affected relationships
affected views
affected drawings
affected validation rules
affected document outputs
```

This allows OneDraft to determine the computational scope of a proposed change before committing it.

32. PRE-COMMIT VALIDATION

Commands may request:

```
pre_commit_validation = true
```

The engine evaluates the proposed model state before finalization.

Conceptually:

```
CURRENT MODEL
↓
PROPOSED MUTATION
↓
TEMPORARY MODEL STATE
↓
```

VALIDATION

↓

PASS / BLOCK

The temporary state must never become current project state unless the command transaction succeeds.

33. VALIDATION GATES

The platform shall support validation gates.

Initial gates:

MODEL_GATE

CODE_GATE

COORDINATION_GATE

DOCUMENT_GATE

FINALIZATION_GATE

ACCEPTANCE_GATE

A gate defines the minimum conditions required before a workflow can proceed.

34. BLOCKING RULES

Rules may be marked:

BLOCKING

NON_BLOCKING

A blocking failure prevents the associated workflow transition.

Example:

FINALIZATION_GATE

may require:

no CRITICAL findings

no unresolved blocking CODE findings

no unresolved MODEL integrity failures

35. VALIDATION POLICY

Validation policy determines how findings affect workflow.

validation.validation_policies

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(255) NOT NULL
description TEXT
profile_id UUID
blocking_severities JSONB
blocking_categories JSONB
required_rules JSONB
waiver_policy JSONB
status VARCHAR(32) NOT NULL
version INTEGER NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

36. WAIVERS

A finding may be waived only through an explicit workflow.

validation.waivers

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
finding_id UUID NOT NULL
requested_by UUID NOT NULL
approved_by UUID
reason TEXT NOT NULL
scope JSONB
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
approved_at TIMESTAMPTZ
expires_at TIMESTAMPTZ
```

A waiver does not delete the finding.

37. ARIA RESOLUTION WORKFLOW

When validation detects a finding, Aria may analyze the finding.

The workflow becomes:

```
FINDING
↓
ARIA ANALYSIS
↓
DESIGN PROPOSAL
↓
IMPACT ANALYSIS
↓
```

COMMAND
↓
MODEL REVISION
↓
VALIDATION

Aria does not directly alter the validation result.

38. AUTOMATED RESOLUTION POLICY

Some findings may permit automated resolution.

Examples:

minor dimension adjustment
annotation conflict
view regeneration
non-critical drawing coordination

Others require explicit review.

Examples:

occupancy change
fire separation change
egress modification
structural system change
major code conflict
professional review requirement

The rule registry shall identify the permitted automation level.

39. RESOLUTION STRATEGIES

Initial resolution types:

AUTO_FIX
ARIA_PROPOSAL
USER_DECISION
PROFESSIONAL_REVIEW
MANUAL_REVIEW
NOT_RESOLVABLE

40. VALIDATION EXECUTION PIPELINE

The complete pipeline is:

```
LOAD PROJECT
↓
LOAD REVISION
↓
LOAD MODEL VERSION
↓
LOAD CODE MANIFEST
↓
LOAD VALIDATION PROFILE
↓
CHECK MODEL INTEGRITY
↓
RESOLVE APPLICABLE RULES
↓
EXECUTE GEOMETRIC VALIDATION
↓
EXECUTE SPATIAL VALIDATION
↓
EXECUTE CODE VALIDATION
↓
EXECUTE COORDINATION VALIDATION
↓
EXECUTE DOCUMENT VALIDATION
↓
AGGREGATE FINDINGS
↓
UPDATE FINDING LIFECYCLES
↓
GENERATE SUMMARY
↓
COMMIT VALIDATION RUN
```

41. ASYNCHRONOUS EXECUTION

Full-project validation executes asynchronously.

Endpoint:

POST /api/v1/projects/{project_id}/validation

Response:

```
{
  "data": {
    "validation_run_id": "uuid",
    "job_id": "uuid",
    "status": "QUEUED"
  }
}
```

42. VALIDATION STATUS

Initial statuses:

QUEUED
RUNNING
PARTIAL
COMPLETED
FAILED
CANCELLED
INVALIDATED

43. VALIDATION JOB

The worker receives:

project_id
revision_id
model_version_id
code_manifest_id
validation_profile_id

The worker must verify that the referenced model version still exists before execution.

44. STALE VALIDATION PROTECTION

If a new revision becomes current while validation is running, the validation run remains historically valid for the revision against which it was executed.

It must not automatically become validation for the newer revision.

Therefore:

Revision 17
↓
Validation Run 41

remains distinct from:

Revision 18
↓
Validation Run 42

45. VALIDATION HASH

At completion the engine generates:

execution_hash

derived from:

model_version_id
code_manifest_id
validation_profile_id
rule_versions
engine_version

This creates reproducibility metadata.

46. VALIDATION SUMMARY

Every completed validation run generates:

total_rules
applicable_rules
passed
failed
warnings
errors
critical_findings
not_applicable
skipped
execution_time

Stored as:

summary JSONB

47. VALIDATION SCORE

The platform may calculate a project validation score for interface purposes.

However:

score != compliance certification

A numerical score must never replace the underlying findings.

The underlying rule-level results remain authoritative.

48. DOCUMENT VALIDATION

Before document generation completes, the engine verifies:

- all required views exist
- all required sheets exist
- drawing references resolve
- sheet references resolve
- revision identifiers are consistent
- model version identifiers are consistent
- required schedules exist
- required notes exist
- required annotations exist

49. DRAWING COORDINATION VALIDATION

Drawing validation shall detect:

- missing views
- stale views
- stale drawings
- incorrect revision references
- incorrect model version references
- orphaned sheet drawings
- duplicate drawing numbers
- missing sheet numbers
- invalid sheet templates

50. CROSS-DRAWING CONSISTENCY

The engine must eventually compare information appearing across drawings.

Examples:

floor plan room number

↔

room schedule

door tag

↔

door schedule

window tag

↔

window schedule

level elevation

↔

section

building height

↔
elevation

Conflicts become coordination findings.

51. SCHEDULE VALIDATION

Schedules are derived from the model and therefore must be validated against current model state.

The engine checks:

schedule source objects
schedule counts
schedule identifiers
schedule values
schedule revision

A schedule must never silently contain data from an obsolete model version.

52. DOCUMENT PACKAGE VALIDATION

Before finalization:

MODEL
↓
VALIDATION
↓
DRAWINGS
↓
DOCUMENT PACKAGE
↓
PACKAGE VALIDATION

The package validator verifies that every file belongs to the same intended:

project
revision
model version
code manifest
validation state

53. VALIDATION GATE FOR DOCUMENT GENERATION

The document engine shall query:

```
GET /api/v1/projects/{project_id}/validation/status
```

The response indicates:

```
READY  
BLOCKED  
WARNING  
INCOMPLETE
```

Document generation may proceed only according to the project's configured validation policy.

54. VALIDATION GATE FOR ACCEPTANCE

Acceptance must identify the validation state presented to the customer.

The acceptance record therefore references:

```
validation_run_id  
revision_id  
model_version_id  
code_manifest_id  
document_package_id
```

This establishes exactly what was accepted.

55. API RESOURCE GROUP

Validation endpoints:

```
POST /projects/{project_id}/validation  
GET /projects/{project_id}/validation  
GET /projects/{project_id}/validation/{run_id}  
GET /projects/{project_id}/validation/{run_id}/results  
GET /projects/{project_id}/validation/findings  
GET /projects/{project_id}/validation/findings/{finding_id}  
POST /projects/{project_id}/validation/findings/{finding_id}/waiver  
POST /projects/{project_id}/validation/findings/{finding_id}/resolve  
GET /projects/{project_id}/validation/status
```

56. VALIDATION REQUEST

POST /api/v1/projects/{project_id}/validation

Request:

```
{
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "code_manifest_id": "uuid",
  "validation_profile_id": "uuid",
  "categories": [
    "CODE",
    "GEOMETRY",
    "SPATIAL",
    "ACCESSIBILITY",
    "COORDINATION",
    "DOCUMENT"
  ]
}
```

57. VALIDATION RESPONSE

```
{
  "data": {
    "validation_run_id": "uuid",
    "job_id": "uuid",
    "revision_id": "uuid",
    "model_version_id": "uuid",
    "status": "QUEUED"
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

58. FINDINGS RESPONSE

```
{
  "data": [
    {
      "finding_id": "uuid",
      "rule_code": "ACCESS-DOOR-001",
      "status": "FAIL",
      "severity": "ERROR",
      "description": "Door clear width does not satisfy the applicable requirement.",
      "affected_objects": [
        "uuid"
      ]
    }
  ]
}
```

```

    ],
    "expected_value": {
      "value": 900,
      "unit": "mm"
    },
    "actual_value": {
      "value": 813,
      "unit": "mm"
    }
  }
]
}

```

59. COORDINATION QUERY

The engine shall support targeted coordination.

POST /api/v1/projects/{project_id}/validation/coordination

Request:

```

{
  "revision_id": "uuid",
  "objects": [
    "uuid",
    "uuid"
  ],
  "domains": [
    "ARCHITECTURAL",
    "STRUCTURAL",
    "MECHANICAL"
  ]
}

```

This permits rapid local validation without necessarily executing a complete project validation.

60. LOCAL VALIDATION

When Aria modifies a small portion of the model, OneDraft may execute targeted validation first.

Example:

```

MOVE WALL
↓
AFFECTED OBJECTS
↓
LOCAL VALIDATION
↓
PASS
↓
COMMIT

```

A subsequent full-project validation may still be required.

61. VALIDATION DEPENDENCY GRAPH

Each rule may declare dependencies:

```
requires:  
geometry  
occupancy  
level  
space  
code_manifest
```

The engine resolves dependencies before evaluation.

If a dependency is unavailable:

```
status = SKIPPED
```

rather than an incorrect:

```
PASS
```

62. RULE EXECUTION ORDER

The default order is:

```
MODEL_INTEGRITY  
↓  
GEOMETRY  
↓  
SPATIAL  
↓  
SITE  
↓  
LIFE_SAFETY  
↓  
ACCESSIBILITY  
↓  
CODE  
↓  
SYSTEM_COORDINATION  
↓  
DOCUMENT
```

Dependencies may alter this order when necessary.

63. ERROR ISOLATION

A failure in one rule must not corrupt the entire validation run.

Conceptually:

```
RULE A → PASS  
RULE B → FAIL  
RULE C → ERROR  
RULE D → PASS
```

The run continues unless the error represents a systemic engine failure.

64. ENGINE FAILURE

A validation run becomes:

FAILED

when the engine cannot reliably establish the requested validation state.

Examples:

```
missing model version  
corrupt geometry reference  
unavailable code manifest  
internal rule execution failure  
database transaction failure
```

A failed run is never interpreted as:

PASS

65. VALIDATION CACHE

Future optimization may cache deterministic validation results.

Cache identity may use:

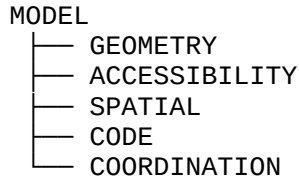
```
model_version_id  
rule_id  
rule_version  
code_manifest_id  
engine_version
```

Cached results must remain provenance-identical to directly computed results.

66. PARALLEL RULE EXECUTION

Independent rules may execute in parallel.

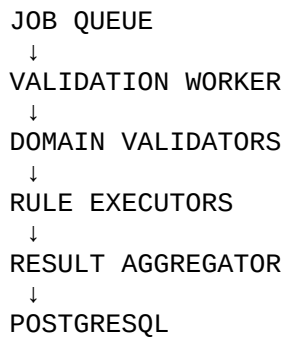
Example:



The final aggregation waits for all required branches.

67. WORKER ARCHITECTURE

The validation worker operates as:

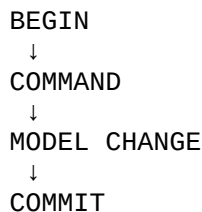


The worker does not directly modify the project model.

68. TRANSACTION BOUNDARY

Validation execution is computationally separate from model mutation.

The model transaction:



Validation:

LOAD_COMMITTED_STATE

↓
EXECUTE
↓
STORE RESULTS

This prevents a long-running validation operation from unnecessarily locking the project model.

69. EVENT RECORDING

The engine shall emit events:

VALIDATION_REQUESTED
VALIDATION_STARTED
VALIDATION_COMPLETED
VALIDATION_FAILED
FINDING_CREATED
FINDING_RESOLVED
FINDING_WAIVED

These events are recorded in:

`audit.project_events`

70. ARIA INTERFACE

Aria may query:

validation status
validation findings
finding history
affected objects
applicable rules
resolution history

Aria may propose:

design changes
command sequences
alternative solutions

Aria may not directly mark a finding:

RESOLVED

without the underlying model change and revalidation.

71. ARIA FINDING ANALYSIS

The Aria interface may receive:

```
{
  "finding_id": "uuid",
  "rule": {},
  "affected_objects": [],
  "model_context": {},
  "constraints": {},
  "project_requirements": {}
}
```

Aria returns a structured proposal:

```
{
  "analysis": "...",
  "proposed_changes": [],
  "estimated_impacts": [],
  "required_review": false
}
```

The proposal becomes:

```
model.design_proposals
```

72. RESOLUTION CONFIRMATION

A resolution is complete only when:

```
PROPOSAL
↓
COMMAND
↓
NEW REVISION
↓
VALIDATION
↓
FINDING ABSENT
```

The finding is then marked:

```
RESOLVED
```

73. REGRESSION DETECTION

If a previously resolved finding returns:

```
finding_hash
```

the engine identifies it as:

REGRESSION

The finding history remains continuous.

74. REGRESSION EXAMPLE

Revision 10

↓

Door width FAIL

Revision 11

↓

Door width corrected

↓

RESOLVED

Revision 12

↓

Wall modification reduces opening

↓

Door width FAIL

↓

REGRESSION

This provides historical intelligence for Aria.

75. VALIDATION HISTORY

The user interface shall eventually expose:

Validation Run 1

12 failures

7 warnings

Validation Run 2

4 failures

3 warnings

Validation Run 3

0 blocking failures

2 warnings

The underlying result records remain immutable.

76. PROJECT READINESS

The engine shall expose a derived project readiness state.

Initial states:

NOT_VALIDATED
VALIDATION_IN_PROGRESS
BLOCKED
CONDITIONALLY_READY
READY_FOR_DOCUMENTATION
READY_FOR_REVIEW
READY_FOR_ACCEPTANCE

This state is derived from validation results and workflow policy.

77. READINESS IS NOT APPROVAL

The system must distinguish:

OneDraft validation

from:

authority approval

and:

professional certification

The platform can report computational validation status but cannot substitute for the authority or professional process applicable to a project.

78. PROFESSIONAL REVIEW INTEGRATION

If a validation policy requires professional review, the engine creates:

`revisions.professional_reviews`

with the appropriate review scope.

The validation finding remains associated with the review requirement.

79. MUNICIPAL REVIEW INTEGRATION

Municipal approval remains represented through:

`projects.approval_statuses`

The validation engine may determine:

`READY_FOR_SUBMISSION`

but the authority's actual decision remains external project state.

80. DOCUMENTATION HANDOFF

Once the project passes the configured documentation gate:

`VALIDATION`

↓

`DOCUMENTATION ENGINE`

The handoff includes:

`project_id`
`revision_id`
`model_version_id`
`code_manifest_id`
`validation_run_id`

The documentation engine must use these exact identifiers.

81. DRAWING REGENERATION TRIGGER

If a validation-resolving command modifies model objects, the system calculates affected documentation.

Conceptually:

`MODEL CHANGE`

↓

`DEPENDENCY GRAPH`

↓

`AFFECTED VIEWS`

↓

`AFFECTED DRAWINGS`

↓

`AFFECTED SHEETS`

↓

`REGENERATION QUEUE`

82. DOCUMENTATION STALENESS

A drawing becomes:

STALE

when its referenced model version is no longer the current project model version.

The drawing must not silently remain presented as current.

83. VALIDATION STALENESS

Similarly:

Validation Run 17

is valid for its referenced revision but becomes:

NOT_CURRENT

relative to a newer project revision.

Historical validity is preserved.

84. API CONTRACT EXTENSION

The OpenAPI contract must add:

ValidationRun
ValidationResult
ValidationFinding
ValidationProfile
ValidationRule
ValidationPolicy
ValidationWaiver
ValidationStatus
CoordinationConflict

85. DOMAIN TYPES

The Python domain layer shall include:

ValidationRun

ValidationResult
ValidationFinding
ValidationRule
ValidationProfile
ValidationPolicy
ValidationWaiver
ValidationSummary
ValidationGate
CoordinationConflict

Enums include:

ValidationStatus
ValidationResultStatus
Severity
FindingStatus
ResolutionType
ValidationCategory

86. SERVICE INTERFACES

The initial service interfaces shall be:

ValidationService
RuleRegistry
RuleExecutor
FindingService
CoordinationService
ImpactAnalysisService
ValidationGateService

87. VALIDATION SERVICE

Conceptually:

```
class ValidationService:
    def create_run(...):
        ...

    def execute_run(...):
        ...

    def get_results(...):
        ...

    def get_findings(...):
        ...

    def evaluate_gate(...):
        ...
```

88. RULE EXECUTOR

Conceptually:

```
class RuleExecutor:
    def resolve_applicability(...):
        ...

    def execute(...):
        ...

    def build_result(...):
        ...
```

The executor must operate against immutable model context.

89. COORDINATION SERVICE

Conceptually:

```
class CoordinationService:
    def analyze_objects(...):
        ...

    def detect_conflicts(...):
        ...

    def calculate_impact(...):
        ...
```

The service consumes the model dependency graph and geometry subsystem.

90. VALIDATION GATE SERVICE

Conceptually:

```
class ValidationGateService:
    def evaluate_model_gate(...):
        ...

    def evaluate_document_gate(...):
        ...

    def evaluate_finalization_gate(...):
```

```
...  
def evaluate_acceptance_gate(...):  
    ...
```

91. FIRST AUTOMATED VALIDATION TEST

The first complete test shall execute:

1. Create project
2. Create building
3. Create levels
4. Create spaces
5. Create walls
6. Create doors
7. Create windows
8. Generate geometry
9. Attach code manifest
10. Create validation profile
11. Run model integrity validation
12. Run geometric validation
13. Run spatial validation
14. Run accessibility validation
15. Run code validation
16. Run coordination validation
17. Aggregate findings
18. Resolve one finding through an Aria proposal
19. Execute the resulting model command
20. Create new revision
21. Revalidate
22. Detect resolved finding
23. Confirm no regression

- 24. Evaluate documentation gate
 - 25. Generate validation summary
 - 26. Record complete validation event history
-

92. MVP SUCCESS CRITERIA

The Validation & Coordination Engine is successful when it can:

Load an exact model version

Load an applicable code manifest

Determine applicable rules

Execute deterministic validation rules

Detect geometry conflicts

Detect spatial conflicts

Detect coordination conflicts

Record findings

Preserve finding history

Track resolutions

Detect regressions

Evaluate workflow gates

Provide structured results to Aria

Prevent false PASS states

Maintain complete provenance

93. INTEGRATION SUCCESS CRITERIA

The complete OneDraft workflow must now support:

PROJECT

↓

REQUIREMENTS

↓

COMPUTATIONAL MODEL

↓

PARAMETRIC GEOMETRY

↓
CODE MANIFEST
↓
VALIDATION
↓
COORDINATION
↓
ARIA RESOLUTION
↓
NEW REVISION
↓
REVALIDATION
↓
DOCUMENTATION

94. ARCHITECTURAL DECISION

Validation is not a final button labelled:

CHECK

It is a continuous computational subsystem attached to the versioned project model.

The system therefore operates as:

MODEL
+
RULES
+
CONSTRAINTS
+
GEOMETRY
+
REGULATORY MANIFEST
+
DEPENDENCY GRAPH
↓
VALIDATION STATE

That state becomes another derived but fully traceable component of the project record.

95. ONE-DRAFT COMPUTATIONAL LOOP

The OneDraft architecture now establishes the following continuous loop:

USER REQUIREMENT
↓
ARIA INTERPRETATION
↓
PROJECT MODEL
↓

PARAMETRIC GEOMETRY
↓
REGULATORY ANALYSIS
↓
VALIDATION
↓
COORDINATION
↓
ARIA ANALYSIS
↓
DESIGN PROPOSAL
↓
MODEL COMMAND
↓
NEW REVISION
↓
REVALIDATION
↓
DRAWING GENERATION
↓
DOCUMENT PACKAGE
↓
CUSTOMER REVIEW
↓
ACCEPTANCE

The loop continues until the project reaches the configured workflow state.

96. CURRENT ONEDRAFT STACK

The OneDraft implementation stack now consists of:

- 01 — PRODUCT DEFINITION
- 02 — SYSTEM ARCHITECTURE
- 03 — TECHNOLOGY ARCHITECTURE
- 04 — DOMAIN MODEL
- 05 — DATABASE / OPENAPI CONTRACT
- 06 — PROJECT MODEL ENGINE
- 07 — GEOMETRY & PARAMETRIC MODEL ENGINE
- 08 — COMPLIANCE & REGULATORY ENGINE
- 09 — VALIDATION & COORDINATION ENGINE

The architecture now has a computational path from:

USER SPECIFICATION

to:

VALIDATED PROJECT MODEL

rather than merely from:

PROMPT

to:

DRAWING

97. NEXT STACK LAYER

The next implementation layer is:

DOCUMENTATION & DRAWING GENERATION ENGINE

This layer consumes:

validated project model
+
geometry
+
views
+
validation state
+
code manifest
+
project graphic standards

and produces:

plans
elevations
sections
details
schedules
annotations
dimensions
sheets
ARCH D documentation

while maintaining:

model_version_id
revision_id
validation_run_id
code_manifest_id

throughout the generation pipeline.

98. NEXT IMPLEMENTATION ARTIFACTS

The next engineering package shall consist of:

```
validation_schema.sql
validation_types.py
validation_service.py
rule_registry.py
coordination_service.py
validation_worker.py
validation_rules/
    model_integrity/
    geometry/
    spatial/
    accessibility/
    life_safety/
    code/
    coordination/
    documentation/
tests/
    test_validation_pipeline.py
    test_rule_execution.py
    test_finding_lifecycle.py
    test_regression_detection.py
    test_validation_gates.py
```

The OpenAPI contract shall be extended with the Validation & Coordination resources.

99. FINAL ARCHITECTURAL DECISION

OneDraft shall not generate drawings and then attempt to determine whether the drawings are correct.

Instead:

```
COMPUTATIONAL PROJECT MODEL
    ↓
PARAMETRIC GEOMETRY
    ↓
REGULATORY MANIFEST
    ↓
VALIDATION
    ↓
COORDINATION
    ↓
VALIDATED MODEL STATE
```

↓
DRAWING GENERATION

The architectural documentation is therefore generated from a validated computational state.

This establishes the necessary separation between:

DESIGN GENERATION

and:

DESIGN VERIFICATION

while allowing both to operate continuously within the same version-controlled project environment.

100. SPECIFICATION STATUS

ONEDRAFT VALIDATION & COORDINATION ENGINE v0.1

STATUS: ESTABLISHED

The OneDraft stack has now progressed through:

PERSISTENCE
↓
API
↓
PROJECT MODEL
↓
GEOMETRY
↓
PARAMETRIC MODELING
↓
COMPLIANCE
↓
REGULATORY ANALYSIS
↓
VALIDATION
↓
COORDINATION

.